

Fundamentos de Git y GitHub

Del ciclo local al trabajo con ramas

Desarrollo Web Backend · 1 sesión · Gestión 02/2026 · M. Sc. Huáscar F. Gonzales G.

Nivel básico · CLI + GitHub web · Requisito: GL01 completada

Contenido

- 01 Identificación de la práctica
- 02 Objetivos
- 03 Antes de empezar
- 04 Configuración inicial (una sola vez)
- 05 Ejercicio 1 — El ciclo local completo
- 06 Ejercicio 2 — Deshacer en tres niveles
- 07 Ejercicio 3 — Ramas: crear, trabajar y fusionar
- 08 Ejercicio de trabajo independiente
- 09 Comandos de referencia
- 10 Criterios de evaluación del laboratorio
- 11 Bibliografía y recursos

01 Identificación de la práctica

Campo	Detalle
Facultad	Ciencias Puras
Carrera	Ingeniería Informática
Asignatura	Desarrollo Web Backend
Sigla y código	INF 560
Eje	Transversal — Control de versiones
Nº de guía	Guía de Laboratorio Nº 02
Tema	Fundamentos de Git y GitHub: ciclo local, deshacer cambios y trabajo con ramas
Requisito	GL01 completada (Git, cuenta de GitHub y autenticación SSH configuradas)
Duración estimada	1 sesión
Docente	M. Sc. Huáscar Fedor Gonzales Guzmán

02 Objetivos

Objetivo general

Aplicar Git en el flujo de trabajo local y su publicación en GitHub, dominando el ciclo de confirmación de cambios, las operaciones para deshacer y el trabajo con ramas, sobre un mismo proyecto que crece de forma coherente a lo largo de la práctica.

Objetivos específicos

- Configurar la identidad de Git y comprender la diferencia entre `commit` (local) y `push` (remoto).
- Ejecutar el ciclo `working directory` → `staging` → `repository` y leer el historial como una línea de tiempo.
- Excluir archivos del control de versiones mediante `.gitignore`.
- Deshacer cambios de forma correcta según dónde estén: en el directorio de trabajo, en `staging` o ya confirmados.
- Crear ramas para aislar el trabajo, fusionarlas en `main` y comprender la fusión `fast-forward`.
- Consolidar todo lo anterior en un ejercicio de trabajo independiente publicado en GitHub.

03 Antes de empezar

Esta guía se trabaja sobre un mismo proyecto que va creciendo ejercicio a ejercicio, de modo que su historial cuente una historia coherente en lugar de ser comandos sueltos. Ejecute los ejercicios en orden: cada uno asume el estado que dejó el anterior.

Definición · Git y GitHub

Git es el sistema de control de versiones que corre en su máquina: registra la historia de sus archivos en un repositorio local. GitHub es una plataforma en la nube que aloja repositorios remotos y añade colaboración (Pull Requests, Issues, revisión de código).

Regla mental: `commit` guarda en su repositorio local; `push` envía esos commits a GitHub. No son lo mismo.

Requisitos previos

- Git instalado. Verifíquelo con `git --version`.
- Una cuenta en `github.com` con autenticación SSH configurada (GL01).
- Manejo básico de terminal (moverse entre carpetas, crear archivos).

04 Configuración inicial (una sola vez)

Git firma cada commit con su nombre y correo. Configúrelos globalmente antes del primer commit para que su historial quede bien atribuido.

terminal

Copiar

```
$ git --version
$ git config --global user.name "Tu Nombre"
$ git config --global user.email "tu@correo.com"
$ git config --global init.defaultBranch main

# Verificar todo lo configurado:
$ git config --list
```

Nota

El correo debería coincidir con el de su cuenta de GitHub para que los commits se asocien a su perfil. `init.defaultBranch main` hace que los repositorios nuevos usen `main` en lugar de `master`.

05 Ejercicio 1 — El ciclo local completo

Objetivo

Entender el flujo working directory → staging → repository y ver el historial como una línea de tiempo. Es la base de todo lo demás.

Pasos

1. Cree la carpeta del proyecto y entre en ella:

```
terminal
```

[Copiar](#)

```
$ mkdir git-practica  
$ cd git-practica
```

2. Inicialice el repositorio:

```
terminal
```

[Copiar](#)

```
$ git init
```

Esto crea una carpeta oculta `.git` donde Git guarda toda la historia. Aún no ha guardado nada, solo preparó el terreno.

3. Cree dos archivos con algo de contenido, por ejemplo un `README.md` y un `index.html` mínimo.

4. Mire el estado. Git verá los archivos como untracked (sin seguimiento):

```
terminal
```

[Copiar](#)

```
$ git status
```

5. Añada el `README` al área de preparación (staging):

```
terminal
```

[Copiar](#)

```
$ git add README.md  
$ git status      # ahora README aparece como "staged"
```

Definición · staging area

El staging area (o índice) es una zona intermedia donde elige qué cambios entrarán en el próximo commit. Le permite confirmar solo una parte de lo que modificó, en vez de todo a la vez.

6. Confirme el primer commit con un mensaje claro:

```
terminal
```

[Copiar](#)

```
$ git commit -m "Agrega README inicial del proyecto"
```

7. Ahora añada y confirme `index.html`, y haga un par de cambios más (edite el HTML, vuelva a add y commit) hasta tener 3 o 4 commits.

```
terminal
```

[Copiar](#)

```
$ git add index.html  # añadir: pone el archivo en el "staging area"  
$ git commit -m "Crear index.html" # confirmar: guarda una foto del estado
```

```
terminal
```

[Copiar](#)

```
# editas index.html en tu editor...
$ git add index.html
$ git commit -m "Agregar título a la página"

# vuelves a editar...
$ git add index.html
$ git commit -m "Agregar párrafo de bienvenida"
```

8. Revise el historial:

```
terminal Copiar

$ git log
$ git log --oneline # versión compacta, una línea por commit
```

Nota · buen mensaje de commit

Escríbalos en modo imperativo y describiendo el qué, no el cómo. Por ejemplo, Corrige enlace roto en la navegación en lugar de cambios varios.

Añada un .gitignore

Algunos archivos no deben versionarse (dependencias, credenciales). Cree un archivo `.gitignore`, confírmelo, y Git dejará de rastrear lo que liste:

```
.gitignore Copiar

node_modules/
.env
*.log
.DS_Store

terminal Copiar

$ git add .gitignore
$ git commit -m "Agrega .gitignore"
```

Advertencia

`.gitignore` solo afecta a archivos aún no rastreados. Si ya confirmó por error un archivo (por ejemplo `.env`), agregarlo al ignore no basta: hay que sacarlo con `git rm --cached .env`.

06 Ejercicio 2 — Deshacer en tres niveles

Objetivo

Practicar cómo revertir cambios según dónde estén: en el directorio de trabajo, en staging o ya confirmados en la historia. Aquí es donde de verdad se aprende, porque implica romper y arreglar.

Nivel A — Descartar cambios sin confirmar

Edite README.md y, sin hacer add, descarte el cambio para volver al último commit:

terminal

Copiar

```
$ git restore README.md
```

Nivel B — Sacar algo de staging

Modifique un archivo, hágale add, y luego arrepéntase: sáquelo del staging sin perder el cambio en disco:

terminal

Copiar

```
$ git add index.html
$ git restore --staged index.html # vuelve a "modified", el cambio sigue ahí
```

Nivel C — Anular un commit ya hecho

Haga un commit con un cambio cualquiera. Hay varias formas de deshacerlo, y elegir bien importa:

terminal

Copiar

```
$ git revert HEAD --no-edit # crea un NUEVO commit que revierte el anterior

$ git reset --soft HEAD~1 # borra el commit, conserva los cambios en staging
$ git reset --mixed HEAD~1 # borra el commit, cambios sin preparar (por defecto)
$ git reset --hard HEAD~1 # borra el commit Y los cambios: no hay vuelta atrás
```

Advertencia · reset reescribe la historia

reset borra commits. Nunca lo use sobre commits que ya subió y comparten otras personas: úselo solo en trabajo local. Para deshacer algo ya publicado, use revert, que suma un commit en vez de borrar.

--hard descarta cambios de forma irrecuperable. Piénselo dos veces.

Comparación rápida

Comando	Qué hace
<code>git restore <archivo></code>	Descarta cambios del directorio de trabajo (vuelve al último commit).
<code>git restore --staged <archivo></code>	Saca el archivo de staging, conservando el cambio en disco.
<code>git revert <commit></code>	Crea un commit nuevo que deshace otro. Seguro para historia compartida.
<code>git reset --soft</code>	Elimina el commit; deja los cambios en staging.
<code>git reset --mixed</code>	Elimina el commit; deja los cambios sin preparar.
<code>git reset --hard</code>	Elimina el commit y los cambios. Irreversible.

07 Ejercicio 3 — Ramas: crear, trabajar y fusionar

Paso 1 — Prepare el proyecto base

Cree una carpeta nueva y arranque el repositorio con un primer commit en `main`:

```
terminal Copiar
$ mkdir practica-ramas
$ cd practica-ramas
$ git init
$ echo "<h1>Mi página</h1>" > index.html
$ git add index.html
$ git commit -m "Crear página inicial"
```

Compruebe dónde está parado con `git branch`. Verá `* main` (el asterisco indica la rama activa). Este es su punto de partida estable.

Paso 2 — Cree una rama y cámbiese a ella

Va a desarrollar un menú de navegación en una rama aparte:

```
terminal Copiar
$ git checkout -b menu-navegacion
```

El `-b` crea la rama y le cambia a ella en un solo paso. Vuelva a correr `git branch` y ahora el asterisco estará en `menu-navegacion`. A partir de aquí, todo lo que haga queda aislado.

Paso 3 — Haga un par de commits en la rama

Edite el HTML y confirme cada cambio:

```
terminal Copiar
$ echo "<nav>Inicio | Contacto</nav>" >> index.html
$ git add index.html
$ git commit -m "Agregar barra de navegación"
$ echo "<footer>2026</footer>" >> index.html
$ git add index.html
$ git commit -m "Agregar pie de página"
```

Ahora la rama `menu-navegacion` tiene 2 commits que `main` no tiene.

Paso 4 — Compruebe que `main` sigue intacta

Aquí está la parte importante. Cámbiese de vuelta a `main` y mire el archivo:

```
terminal Copiar
$ git checkout main
$ cat index.html
```

Verá solo `<h1>Mi página</h1>`. El `<nav>` y el `<footer>` no están. Eso demuestra que su trabajo vivió aislado: `main` nunca se enteró. Confírmelo también con `git log --oneline`: solo aparece el commit inicial.

Paso 5 — Fusione la rama en `main`

Ya que está parado en `main`, traiga los cambios de su rama y vuelva a mirar el archivo:

```
terminal Copiar
```

```
$ git merge menu-navegacion  
$ cat index.html
```

Ahora sí aparece todo: el `<h1>`, el `<nav>` y el `<footer>`. El trabajo que hizo aislado ya forma parte de la línea principal.

Paso 6 — Limpie la rama que ya cumplió su función

```
terminal
```

[Copiar](#)

```
$ git branch -d menu-navegacion
```

La rama era solo un espacio de trabajo temporal; los commits ya viven en `main`, así que borrarla no pierde nada.

Paso 7 — Revise el historial completo

```
terminal
```

[Copiar](#)

```
$ git log --oneline --graph
```

Nota · fast-forward

El `--graph` dibuja la estructura. Verá los tres commits en una sola línea, porque al no haber cambios simultáneos en `main`, Git pudo hacer una fusión limpia, lo que se llama fast-forward.

08 Ejercicio de trabajo independiente

Este ejercicio es individual y sin guía paso a paso: usted debe aplicar por su cuenta todo lo practicado en los Ejercicios 1 a 3 sobre un proyecto nuevo, y publicarlo en GitHub usando la autenticación SSH configurada en la GL01. El objetivo es demostrar dominio del ciclo local, del manejo de ramas y de las operaciones para deshacer.

Enunciado

Construya el repositorio mini-portafolio: un mini sitio web estático (por ejemplo, una página personal) cuyo historial de Git cuente de forma clara cómo fue creciendo el proyecto, y publíquelo en GitHub.

Requisitos obligatorios

- Inicialice un repositorio local mini-portafolio con al menos tres archivos (`index.html`, `styles.css` y `README.md`).
- El historial debe tener como mínimo seis commits atómicos, con mensajes en modo imperativo que describan el qué.
- Incluya un `.gitignore` que excluya al menos `node_modules/`, `.env` y `*.log`, y demuestre que un archivo ignorado no aparece en `git status`.
- Demuestre las tres formas de deshacer: (a) descartar un cambio con `git restore`; (b) sacar algo de staging con `git restore --staged`; (c) revertir un commit ya confirmado con `git revert` (anote el hash revertido).
- Desarrolle una funcionalidad en una rama llamada `feature-galeria` con al menos dos commits; compruebe que `main` queda intacta, fusione la rama en `main` y elimínela.
- Publique el repositorio en GitHub por SSH: cree el repositorio remoto, vincúlelo con `git remote add origin` y suba el trabajo con `git push -u origin main`.

Preguntas de reflexión (inclúyalas en el README)

- ¿Cuál es la diferencia entre `commit` y `push`? ¿Por qué importa?
- ¿Cuándo usaría `git revert` en lugar de `git reset`?
- ¿Qué ventaja le dio trabajar la funcionalidad en una rama separada en lugar de directamente en `main`?

Entregable

- Enlace público al repositorio mini-portafolio en GitHub.
- Carpeta evidencias/ con capturas de: `git log --oneline --graph`, un `git status` que muestre la ausencia del archivo ignorado, y la vista del repositorio en GitHub.
- El README con las tres preguntas de reflexión respondidas.

Consideraciones

No versione credenciales (`.env`) ni dependencias (`node_modules/`): hacerlo descuenta puntaje. El historial debe ser propio y coherente; evite un único commit gigante con todo el trabajo.

09 Comandos de referencia

Comando	Acción
<code>git init</code>	Inicializa un repositorio en la carpeta actual

Comando	Acción
<code>git status</code>	Muestra el estado de los archivos
<code>git add <archivo> / git add .</code>	Prepara cambios (staging)
<code>git commit -m "..."</code>	Confirma los cambios preparados
<code>git log --oneline --graph</code>	Muestra el historial como árbol
<code>git restore <archivo></code>	Descarta cambios del working directory
<code>git restore --staged <archivo></code>	Saca un archivo del staging
<code>git revert <commit></code>	Crea un commit que deshace otro
<code>git checkout -b <rama></code>	Crea una rama y cambia a ella
<code>git merge <rama></code>	Fusiona una rama en la actual
<code>git branch -d <rama></code>	Elimina una rama fusionada
<code>git push -u origin main</code>	Publica y sigue la rama en GitHub

10 Criterios de evaluación del laboratorio

La calificación se basa en el ejercicio de trabajo independiente (Sección 08) y en la correcta ejecución de los ejercicios guiados. Puntaje total: 100 puntos.

Criterio	Descripción	Pts.
Configuración e historial	Repositorio con ≥ 6 commits atómicos y mensajes en imperativo que describen el qué.	20
.gitignore	Archivo .gitignore correcto y evidencia de que un archivo ignorado no aparece en git status.	10
Deshacer cambios	Las tres operaciones evidenciadas: restore, restore --staged y revert (con el hash).	20
Trabajo con ramas	Rama creada, trabajo aislado, main intacta, fusión a main y limpieza de la rama.	20
Publicación en GitHub	Repositorio remoto creado y push de main por SSH; repositorio accesible.	15
Evidencias y entrega	Capturas completas (log, status, GitHub), README con reflexión y entrega en fecha.	15
Total		100

Nota

Versionar credenciales o dependencias, o entregar un historial con un único commit que agrupe todo el trabajo, descuenta puntaje según la gravedad, a criterio del docente.

11 Bibliografía y recursos

- Chacon, S. & Straub, B. (2014). Pro Git (2.^a ed.), capítulos 2 y 3. Apress. <https://git-scm.com/book/es/v2>.
- Documentación oficial de Git: <https://git-scm.com/doc>.
- GitHub Docs — Introducción: <https://docs.github.com/es/get-started>.
- GitHub Docs — Conexión por SSH: <https://docs.github.com/es/authentication/connecting-to-github-with-ssh>.